St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026
Class: BE-IT A/B, Semester: VII

Subject: Secure Application Development Lab

Student Name: Vishal Mahajan Student Roll No: 01

Experiment – 9 : Study and Implement Symmetric and Asymmetric Encryption Algorithm

Aim: To study and Implement symmetric and asymmetric encryption algorithm
a. Symmetric Key Cryptography (Data Encryption Standard)
b. Asymmetric Key Cryptography (Diffie Hellman)

Objectives: After study of this experiment, the student will be able to

- understand what is cryptography
- understand the concept of symmetric and asymmetric cryptography
- understand where encryption/decryption standards are used

Lab objective mapped: ITL703.6 To **apply** secure coding for cryptography

Prerequisite: Basic knowledge of symmetric and asymmetric key cryptography.

Requirements: C/C++/JAVA/PYTHON

Pre-Experiment Theory:

(a) **Symmetric Key Cryptography**

The Data Encryption Standard (DES) is a symmetric key block cipher algorithm used for data encryption and decryption. It was widely used in the past but is now considered outdated and insecure due to its relatively short key length (56 bits). Nonetheless, understanding how DES works can provide insight into the historical development of encryption algorithms.

Key components of the DES algorithm:

Key Generation:

- The original DES uses a 56-bit key, but only 64 bits are used for the key schedule, with the remaining 8 bits being used for error checking and parity.
- The 56-bit key undergoes a permutation and shifting process to produce 16 subkeys, each of 48 bits in length.

Encryption Process:

- DES operates on 64-bit blocks of plaintext and ciphertext.
- The initial step involves permuting the 64-bit plaintext using a fixed permutation table (initial permutation, IP).
- The 64-bit block is then divided into two halves, each 32 bits in size: the left half (L0) and the right half (R0).

- The core of the DES algorithm consists of 16 rounds, where each round uses the corresponding subkey generated during the key schedule.
- In each round, the right half of the data from the previous round (Ri-1) is expanded to 48 bits using an expansion permutation (E-bit selection).
- The expanded right half is then XORed with the current subkey (Ki) to introduce key mixing.
- The XOR result is passed through a series of eight substitution boxes (S-boxes) to perform nonlinear transformations.
- After S-box substitution, a permutation (P-box) is applied to the result.
- The output is then XORed with the left half of the data from the previous round (Li-1).
- The left and right halves are swapped, and the next round begins.

Decryption Process:

- The decryption process in DES is the same as encryption, but the subkeys are used in reverse order.

Final Permutation:

- After completing all 16 rounds, the two halves are combined and undergo a final permutation (inverse initial permutation, IP^-1) to produce the final 64-bit ciphertext.

(b) **Asymmetric Key Cryptography**

Diffie Hellman key exchange is a cryptographic protocol that allows two parties that have no prior knowledge of each other to jointly establish a shared secret key over an insecure communications channel. This key can then be used to encrypt subsequent communications using a symmetric key cipher.

Algorithm:

Alice and Bob, two users who wish to establish secure communications. We can assume that Alice and Bob know nothing about each other but are in contact.

1. Communicating in the clear, Alice and Bob agree on two large positive integers, p and g, where p is a prime number and g is a primitive root mod p.
2. Alice randomly chooses another large positive integer, XA, which is smaller than p. XA will serve as Alice's private key.
3. Bob similarly chooses his own private key, XB.
4. Alice computes her public key, YA, using the formula YA = (g^XA) mod p.
5. Bob similarly computes his public key, YB, using the formula YB = (g^XB) mod p.
6. Alice and Bob exchange public keys over the insecure circuit.
7. Alice computes the shared secret key, k, using the formula k = (YB ^XA) mod p.
8. Bob computes the same shared secret key, k, using the formula k = (YA ^XB) mod p.
9. Alice and Bob communicate using the symmetric algorithm of their choice and the shared secret key, k, which was never transmitted over the insecure circuit.

Procedure:

Insert following codes with output

a. Symmetric Key Cryptography (Data Encryption Standard)

Code:

```
# Initial Permutation Table
IP = [58, 50, 42, 34, 26, 18, 10, 2,
      60, 52, 44, 36, 28, 20, 12, 4,
      62, 54, 46, 38, 30, 22, 14, 6,
      64, 56, 48, 40, 32, 24, 16, 8,
      57, 49, 41, 33, 25, 17, 9,  1,
      59, 51, 43, 35, 27, 19, 11, 3,
      61, 53, 45, 37, 29, 21, 13, 5,
      63, 55, 47, 39, 31, 23, 15, 7]

# Final Permutation Table
FP = [40, 8, 48, 16, 56, 24, 64, 32,
      39, 7, 47, 15, 55, 23, 63, 31,
      38, 6, 46, 14, 54, 22, 62, 30,
      37, 5, 45, 13, 53, 21, 61, 29,
      36, 4, 44, 12, 52, 20, 60, 28,
      35, 3, 43, 11, 51, 19, 59, 27,
      34, 2, 42, 10, 50, 18, 58, 26,
      33, 1, 41,  9, 49, 17, 57, 25]

# Expansion Table
E = [32, 1, 2, 3, 4, 5, 4, 5,
     6, 7, 8, 9, 8, 9, 10, 11,
     12, 13, 12, 13, 14, 15, 16, 17,
     16, 17, 18, 19, 20, 21, 20, 21,
     22, 23, 24, 25, 24, 25, 26, 27,
     28, 29, 28, 29, 30, 31, 32, 1]

# S-boxes (8 total)
S_BOX = [
   # S1
   [[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],
    [0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],
    [4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0],
    [15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]],
   # S2
   [[15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10],
    [3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5],
    [0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15],
    [13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9]],
   # S3
```

```
    [[10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8],
     [13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1],
     [13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7],
     [1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12]],
    # S4
    [[7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15],
     [13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9],
     [10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4],
     [3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14]],
    # S5
    [[2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9],
     [14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6],
     [4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14],
     [11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3]],
    # S6
    [[12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11],
     [10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8],
     [9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6],
     [4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13]],
    # S7
    [[4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1],
     [13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6],
     [1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2],
     [6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12]],
    # S8
    [[13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7],
     [1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2],
     [7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8],
     [2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]]
]

# P-box Permutation Table
P = [16, 7, 20, 21, 29, 12, 28, 17,
     1, 15, 23, 26, 5, 18, 31, 10,
     2, 8, 24, 14, 32, 27, 3, 9,
     19, 13, 30, 6, 22, 11, 4, 25]

# Key schedule tables
PC1 = [57,49,41,33,25,17,9,
       1,58,50,42,34,26,18,
       10,2,59,51,43,35,27,
       19,11,3,60,52,44,36,
       63,55,47,39,31,23,15,
       7,62,54,46,38,30,22,
       14,6,61,53,45,37,29,
       21,13,5,28,20,12,4]
```

```
PC2 = [14,17,11,24,1,5,
       3,28,15,6,21,10,
       23,19,12,4,26,8,
       16,7,27,20,13,2,
       41,52,31,37,47,55,
       30,40,51,45,33,48,
       44,49,39,56,34,53,
       46,42,50,36,29,32]

SHIFTS = [1, 1, 2, 2, 2, 2, 2, 2,
          1, 2, 2, 2, 2, 2, 2, 1]

# Helper Functions

def bytes_to_bits(b: bytes) -> str:
    return ''.join(format(byte, '08b') for byte in b)

def bits_to_bytes(bstr: str) -> bytes:
    return bytes(int(bstr[i:i+8], 2) for i in range(0, len(bstr), 8))

def permute(block: str, table) -> str:
    return ''.join(block[i-1] for i in table)

def left_shift(bits: str, n: int) -> str:
    return bits[n:] + bits[:n]

def xor(a: str, b: str) -> str:
    return ''.join('1' if i != j else '0' for i, j in zip(a, b))

def sbox_substitution(bits48: str) -> str:
    out = ""
    for i in range(8):
        block = bits48[i*6:(i+1)*6]
        row = int(block[0] + block[5], 2)
        col = int(block[1:5], 2)
        out += format(S_BOX[i][row][col], '04b')
    return out

def feistel(right32: str, subkey48: str) -> str:
    expanded = permute(right32, E)
    mixed = xor(expanded, subkey48)
    sboxed = sbox_substitution(mixed)
    return permute(sboxed, P)

def generate_subkeys(key64_bits: str):
```

```
    key56 = permute(key64_bits, PC1)
    c, d = key56[:28], key56[28:]
    subkeys = []
    for shift in SHIFTS:
        c = left_shift(c, shift)
        d = left_shift(d, shift)
        subkeys.append(permute(c + d, PC2))
    return subkeys

# PKCS#7 padding for 8-byte blocks
def pkcs7_pad(data: bytes, block_size: int = 8) -> bytes:
    pad_len = block_size - (len(data) % block_size)
    if pad_len == 0:
        pad_len = block_size
    return data + bytes([pad_len]) * pad_len

def pkcs7_unpad(data: bytes, block_size: int = 8) -> bytes:
    if not data or len(data) % block_size != 0:
        raise ValueError("Invalid padded data.")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > block_size:
        raise ValueError("Bad padding length.")
    if data[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("Invalid padding bytes.")
    return data[:-pad_len]

#Block Encrypt/Decrypt

def des_encrypt_block(block64_bits: str, subkeys):
    block = permute(block64_bits, IP)
    left, right = block[:32], block[32:]
    for k in subkeys:
        left, right = right, xor(left, feistel(right, k))
    return permute(right + left, FP)

def des_decrypt_block(block64_bits: str, subkeys):
    block = permute(block64_bits, IP)
    left, right = block[:32], block[32:]
    for k in reversed(subkeys):
        left, right = right, xor(left, feistel(right, k))
    return permute(right + left, FP)

# High-Level API (any length)

def normalize_key_to_8_bytes(key_str: str) -> bytes:
    # UTF-8 encode; pad/truncate to 8 bytes (DES expects 8 bytes incl. parity)
```

```
    kb = key_str.encode('utf-8')
    if len(kb) < 8:
        kb = kb + b'\x00' * (8 - len(kb))
    else:
        kb = kb[:8]
    return kb

def des_encrypt_ecb(plaintext: bytes, key_str: str) -> bytes:
    key_bytes = normalize_key_to_8_bytes(key_str)
    key_bits = bytes_to_bits(key_bytes)
    subkeys = generate_subkeys(key_bits)

    padded = pkcs7_pad(plaintext, 8)
    out_bits = ""
    for i in range(0, len(padded), 8):
        block = padded[i:i+8]
        block_bits = bytes_to_bits(block)
        out_bits += des_encrypt_block(block_bits, subkeys)
    return bits_to_bytes(out_bits)

def des_decrypt_ecb(ciphertext: bytes, key_str: str) -> bytes:
    key_bytes = normalize_key_to_8_bytes(key_str)
    key_bits = bytes_to_bits(key_bytes)
    subkeys = generate_subkeys(key_bits)

    if len(ciphertext) % 8 != 0:
        raise ValueError("Ciphertext length must be multiple of 8 bytes.")
    out_bits = ""
    for i in range(0, len(ciphertext), 8):
        block = ciphertext[i:i+8]
        block_bits = bytes_to_bits(block)
        out_bits += des_decrypt_block(block_bits, subkeys)
    plain_padded = bits_to_bytes(out_bits)
    return pkcs7_unpad(plain_padded, 8)

# Run

if __name__ == "__main__":
    message = input("Enter message (any length): ")
    key = input("Enter key (any length, will use first 8 bytes): ")

    ciphertext = des_encrypt_ecb(message.encode('utf-8'), key)
    decrypted = des_decrypt_ecb(ciphertext, key)

    print("\nCiphertext (hex):", ciphertext.hex().upper())
    print("Decrypted text   :", decrypted.decode('utf-8', errors='replace'))
```

Output:



b. Asymmetric Key Cryptography (Diffie Hellman)

Code:

```
def mod_exp(base, exp, mod):
    result = 1
    base = base % mod
    while exp > 0:
        if exp % 2 == 1:  # If exponent is odd
            result = (result * base) % mod
        exp //= 2
        base = (base * base) % mod
    return result

# Step 1: Public values
p = int(input("Enter a prime number (p): "))
g = int(input("Enter a primitive root modulo p (g): "))

# Step 2: Private keys
a = int(input("Enter Alice's private key (a): "))
b = int(input("Enter Bob's private key (b): "))

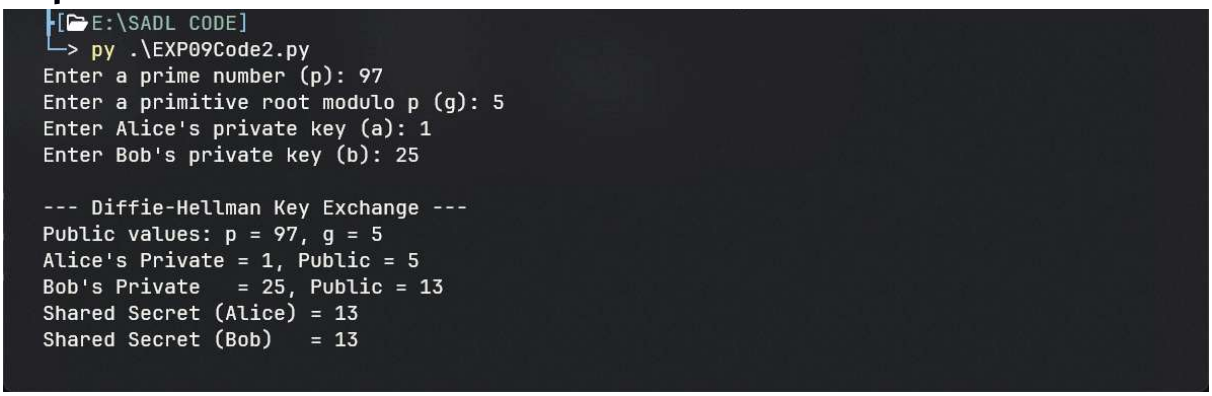
# Step 3: Public keys
A = mod_exp(g, a, p)  # Alice's public key
B = mod_exp(g, b, p)  # Bob's public key

# Step 4: Shared secret
secret_key_alice = mod_exp(B, a, p)
secret_key_bob = mod_exp(A, b, p)

# Step 5: Output
print("\n--- Diffie-Hellman Key Exchange ---")
print(f"Public values: p = {p}, g = {g}")
print(f"Alice's Private = {a}, Public = {A}")
print(f"Bob's Private   = {b}, Public = {B}")
print(f"Shared Secret (Alice) = {secret_key_alice}")
```

```
print(f"Shared Secret (Bob)   = {secret_key_bob}")
```

Output:



Post-Experiments Exercise:

Extended Theory: Nil

Post Experimental Exercise:

Questions:

- Compare symmetric and asymmetric encryption algorithms.
- Discuss the security of DES in the modern context, considering the advancements in cryptanalysis techniques and computing power.
- Discuss real-world applications of the Diffie-Hellman algorithm in various cryptographic protocols, such as secure key exchange for SSL/TLS in web browsers or VPN connections.

Conclusion:

- Write what was performed in the experiment.
- Write the significance of the topic studied in the experiment.

References:

http://cse29-iiith.vlabs.ac.in/